

在真实环境中选择最佳的测试策略

核心问题

模型训练完成后，是固定参数直接泛化，还是在新数据反馈中持续学习？如果持续学习，是否需要保留 Adam 等优化器历史状态？

本次实验希望回答

- 部署后是否需要继续更新
- 继续更新时是否保留优化器状态
- 最终独立测试集上的真实精度

Train 训练初始模型

Validation 选择策略

Test 最终评估

原则：验证集负责选择方案，测试集只用于最终报告结果，避免测试集泄漏。

三种候选策略只改变部署阶段的更新方式

策略一

固定参数泛化

训练后参数不再变化，直接预测新样本。

最稳定，接近传统测试方式。

策略二

仅参数继续更新

保存训练后的模型参数，在新数据反馈中继续学习。

适合数据分布会变化的场景。

策略三

参数 + 优化器状态

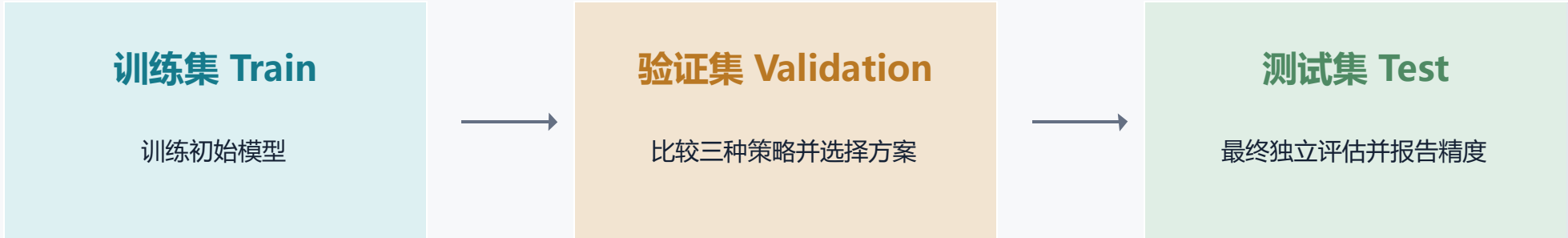
同时保存模型参数与 Adam 一阶矩、二阶矩等状态。

保留训练惯性，可能让持续学习更平滑。

先预测
再根据反馈更新

避免模型提前看到答案

数据划分承担不同角色，测试集只使用一次



禁止在测试集上挑方法

推荐原则

如果数据天然存在时间顺序，优先按时间切分：早期数据训练，中期数据验证，后期数据测试。这样更接近真实上线后的未知数据。

验证集负责决策，测试集负责验收。

验证集选择“仅参数继续更新”作为最佳策略

Validation precision



验证集最佳

策略二

仅参数继续更新

解读：在线更新整体优于固定参数；但保留 Adam 历史状态没有在验证集上带来额外优势。

测试集上三种策略全部输出，策略二仍保持最高

Test precision

策略一：固定参数



0.782180

策略二：仅参数继续更新



0.783197

策略三：参数 + Adam 状态



0.782810

测试集最高

0.783197

验证集推荐策略

在测试集仍为最高

解读：验证集选出的策略二在测试集上也表现最好，说明选择逻辑与独立测试结果一致。

问题1（应用场景，在最高法验证测试）：

如果不区分省份，整个测试集统一采用一种部署策略，三种策略里哪种最好？

结果：在整体测试集口径下，统一使用“仅参数继续更新”略好。

问题2（应用场景，在地区法院验证测试）：

如果按省份分别看，每个省份到底应该：

1. 不更新参数；
2. 仅参数继续更新；
3. 参数 + Adam 状态继续更新？

three_strategy_experiment.py

全量数据先按原始顺序整体切成 Train / Validation / Test = 60% / 20% / 20%。全国基础模型只在 Train_all 上训练，默认 --epochs=30，即训练集会被反复训练 30 轮。随后在验证集比较三种部署策略，并在测试集输出三种策略精度。

province_offline_finetune_experiment.py

默认沿用同一个整体 6:2:2 切分。然后从 Train_all、Val_all、Test_all 中按省份筛选，形成每个省自己的省级训练集、省级验证集、省级测试集。分省份微调默认 --finetune-epochs=30，并用该省验证集选择最佳 epoch / 策略。

分省份离线多轮微调带来明显提升

固定参数加权测试精度

0.782180

分省份离线微调后

0.798290

整体加权提升

+0.016110

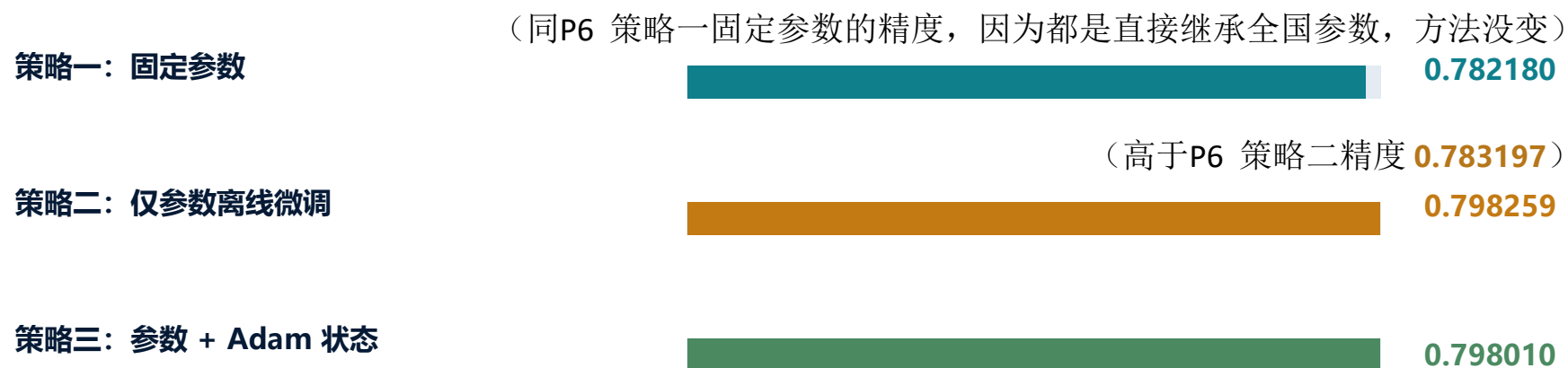
省份层面

27 个省份上升，3 个省份下降，1 个省份持平。

结论：允许多轮离线微调后，分省份模型明显优于直接继承全国权重。

最终测试精度显示 “策略二” 略优于策略三

Test precision



单一策略最高

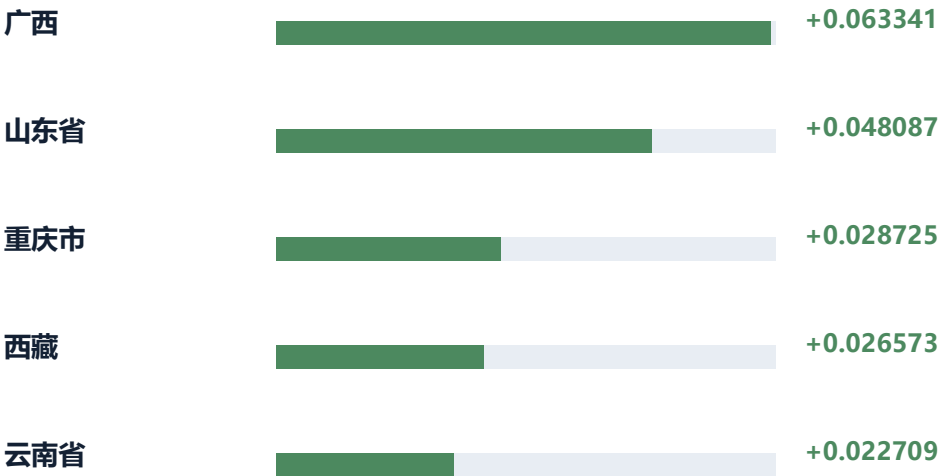
策略二

仅参数离线微调

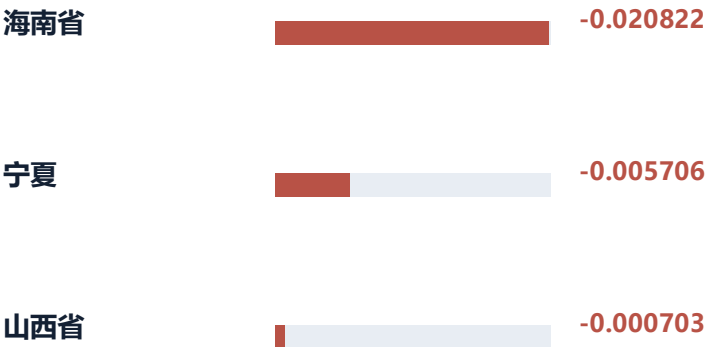
解读：策略二测试集加权精度最高；策略三非常接近，但没有超过策略二。

广西、山东、重庆、西藏、云南提升最明显

提升 Top 5



下降省份



测试方案

1. 全国历史数据 → 训练全国基础模型
2. 目标省份历史数据 → 按时间切分为 训练 / 验证 / 测试
3. 目标省份训练集 → 做省份微调
4. 目标省份验证集 → 选策略和最佳 epoch
5. 目标省份测试集 → 最终报告精度

“验证集选择最佳 epoch”意思是：

模型在某个省份上微调很多轮，比如 `finetune_epochs=20`，不是默认用第 20 轮的模型，而是每训练完一轮，就在该省份的验证集上测一次精度：

epoch 0: 验证精度 0.78
epoch 1: 验证精度 0.80
epoch 2: 验证精度 0.79
epoch 3: 验证精度 0.82 ← 最好
...
epoch 20: 验证精度 0.77

那就保存 epoch 3 的模型参数，最后拿这个模型去测试集评估。

为什么这么做：防止微调过头。省份样本通常比较少，训练轮数多了可能把该省训练集记住，测试集反而变差。验证集用来决定“停在哪一轮比较好”。

测试方案

三个策略的测试流程

对每个目标省份，都执行三种策略：

策略一：固定全国参数

全国模型 → 直接预测该省测试集

策略二：只用全国参数初始化，Adam 状态重新开始

全国模型参数 → 省份训练集微调

每个 epoch 后在省份验证集评估

选择验证集最好的 epoch

用该 epoch 模型评估省份测试集

策略三：全国参数 + 全国 Adam 状态一起继承

全国模型参数 + 全国优化器状态 → 省份训练集继续微调

每个 epoch 后在省份验证集评估

选择验证集最好的 epoch

用该 epoch 模型评估省份测试集

最终策略选择规则

在验证集上比较策略一、策略二、策略三
选验证集精度最高的策略, 把这个策略用于测试集

例如：

山东省验证集：

策略一 0.81

策略二 **0.84**

策略三 0.83

选择策略二

北京市验证集：

策略一 **0.85**

策略二 0.82

策略三 0.80

选择策略一，即用全国参数

代码结果解释

```
运行 province_offline_finetune_experiment (1) × three_strategy_experiment (1) ×

Epoch 018/30 | train_precision=0.771988 | loss=49.681190
Epoch 019/30 | train_precision=0.772430 | loss=49.443430
Epoch 020/30 | train_precision=0.773255 | loss=49.231292
Epoch 021/30 | train_precision=0.774017 | loss=49.222362
Epoch 022/30 | train_precision=0.773875 | loss=49.443986
Epoch 023/30 | train_precision=0.774455 | loss=49.145269
Epoch 024/30 | train_precision=0.776172 | loss=48.906291
Epoch 025/30 | train_precision=0.777934 | loss=48.619159
Epoch 026/30 | train_precision=0.779372 | loss=48.870736
Epoch 027/30 | train_precision=0.781050 | loss=48.601326
Epoch 028/30 | train_precision=0.783106 | loss=48.302784
Epoch 029/30 | train_precision=0.785578 | loss=47.702777
Epoch 030/30 | train_precision=0.786972 | loss=47.525606

验证集三种策略结果
-----
策略一_固定参数: 0.782521
策略二_仅参数继续更新: 0.784515
策略三_参数加优化器状态继续更新: 0.783283
最佳策略: 策略二_仅参数继续更新 | precision=0.784515

测试集三种策略结果
-----
策略一_固定参数: 0.782180
策略二_仅参数继续更新: 0.783197
策略三_参数加优化器状态继续更新: 0.782810
最佳策略: 策略二_仅参数继续更新 | precision=0.783197

验证集最佳策略: 策略二_仅参数继续更新
结果表已保存: outputs\three_strategy_experiment\three_strategy_summary.xlsx
基础模型参数和 Adam 状态已保存: outputs\three_strategy_experiment\base_model_and_adam_state.pth
JSON 摘要已保存: outputs\three_strategy_experiment\three_strategy_summary.json
```

对应P4的精度

Test precision

策略一: 固定参数

0.782180

策略二: 仅参数继续更新

0.783197

策略三: 参数 + Adam 状态

0.782810

代码结果解释

```
运行 province_offline_finetune_experiment (1) × three_strategy_experiment (1) ×

[1/31] 处理省份: 上海市
验证最佳=策略一_固定参数; 固定测试=0.755702; 选中测试=0.755702; delta=+0.000000; 策略二epoch=0; 策略三epoch=0

[2/31] 处理省份: 云南省
验证最佳=策略三_参数加Adam状态离线微调; 固定测试=0.790402; 选中测试=0.813112; delta=+0.022709; 策略二epoch=27; 策略三epoch=28

[3/31] 处理省份: 内蒙古自治区
验证最佳=策略三_参数加Adam状态离线微调; 固定测试=0.776052; 选中测试=0.776570; delta=+0.000518; 策略二epoch=0; 策略三epoch=1

[4/31] 处理省份: 北京市
验证最佳=策略二_仅参数离线微调; 固定测试=0.798819; 选中测试=0.804746; delta=+0.005926; 策略二epoch=30; 策略三epoch=27

[5/31] 处理省份: 吉林省
验证最佳=策略二_仅参数离线微调; 固定测试=0.754215; 选中测试=0.774184; delta=+0.019969; 策略二epoch=25; 策略三epoch=29

[6/31] 处理省份: 四川省
验证最佳=策略三_参数加Adam状态离线微调; 固定测试=0.787798; 选中测试=0.805659; delta=+0.017861; 策略二epoch=28; 策略三epoch=24

[7/31] 处理省份: 天津市
验证最佳=策略三_参数加Adam状态离线微调; 固定测试=0.779740; 选中测试=0.798607; delta=+0.018860; 策略二epoch=27; 策略三epoch=27

[8/31] 处理省份: 宁夏回族自治区
验证最佳=策略二_仅参数离线微调; 固定测试=0.824054; 选中测试=0.818348; delta=-0.005706; 策略二epoch=19; 策略三epoch=20

[9/31] 处理省份: 安徽省
验证最佳=策略三_参数加Adam状态离线微调; 固定测试=0.825533; 选中测试=0.842752; delta=+0.017219; 策略二epoch=30; 策略三epoch=22

[10/31] 处理省份: 山东省
验证最佳=策略二_仅参数离线微调; 固定测试=0.764805; 选中测试=0.812892; delta=+0.048087; 策略二epoch=25; 策略三epoch=21

[11/31] 处理省份: 山西省
验证最佳=策略三_参数加Adam状态离线微调; 固定测试=0.786149; 选中测试=0.785445; delta=-0.000703; 策略二epoch=30; 策略三epoch=26

[12/31] 处理省份: 广东省
```

不同省市在验证集上选择最佳的策略

代码结果解释

对应P7的精度

Test precision

策略一：固定参数

(同P6 策略一固定参数的精度，因为都是直接继承全国参数，方法没变)

0.782180

策略二：仅参数离线微调

(高于P6 策略二精度 0.783197)

0.798259

策略三：参数 + Adam 状态

0.798010

分省份离线微调实验汇总

三个策略最终测试集加权精度与相对策略一提升：

策略一_固定参数：精度=0.782180，提升=+0.000000

策略二_仅参数离线微调：精度=0.798259，提升=+0.016079

策略三_参数加Adam状态离线微调：精度=0.798010，提升=+0.015830

验证集选出的最终策略组合：精度=0.798290，相对策略一提升=+0.016110

上升省份数：27

下降省份数：3

持平省份数：1

验证集最佳策略胜出次数：

验证集最佳策略

策略二_仅参数离线微调 16

策略三_参数加Adam状态离线微调 14

策略一_固定参数 1

对每个省份分别做：

看该省验证集上：策略一、策略二、策略三哪个精度最高，选择这个策略，然后用这个被选中的策略去该省测试集上评估，最后把所有省份的测试结果按测试样本数加权平均，得到：最终组合测试精度

上升 / 下降 / 持平省份数： 对每个省份比较：

验证集选出的策略在测试集上的精度 vs 策略一固定参数在测试集上的精度

如果更高，就是“上升”；更低，就是“下降”；一样就是“持平”。

所以这里表示：

31 个省份中

27 个省份：验证集选出的策略在测试集上优于策略一

3 个省份：验证集选出的策略在测试集上差于策略一

1 个省份：两者持平

一共有 31 个省份参与评估：

- 16 个省份：验证集上策略二精度最高
- 14 个省份：验证集上策略三精度最高
- 1 个省份：验证集上策略一精度最高

饱和神经网络筛选测试集后的分省份微调评估

原始测试集

无需剔除: $RAD=86.73\%$

17,518

86.73%

样本数

实验目标: 先用“纯 $NN + SaturatedClamp$ ”对测试样本排序, 再在保留子集上评估“机理 + NN + 分省份微调”流程。

Q1: 用饱和 NN 筛选测试集, 能否说明这是一个“好测试集”?

不能直接这样表述。

- 若用 *oracle_precision*: 排序时已经看了测试真实标签, 本质是事后筛选难样本。
- 它可以回答“剔除多少低质量/高误差样本后, 指标能达到阈值”。
- 不能证明原始测试集本身更客观、更代表真实泛化能力。

推荐表述: 这是“筛选强度—预测精度”的敏感性实验, 不是对测试集质量的独立证明。

整体实验流程

先筛选, 再在保留测试子集上跑机理 + NN + 分省份微调

数据划分
Train / Val / Test
默认 60% / 20% / 20%

纯饱和 NN
训练 NN
验证集选最优模型
(early stop)

测试集排序
按 *oracle_precision*
打分

逐步剔除
0%, 5%, ..., 80%
保留高分样本

分省份微调评估
按验证集选策略
输出 RA / RAD

步骤 1: 纯 *NN* + *SaturatedClamp* 筛选测试样本

筛选器的作用是给测试样本排序，而不是直接替代最终预测模型

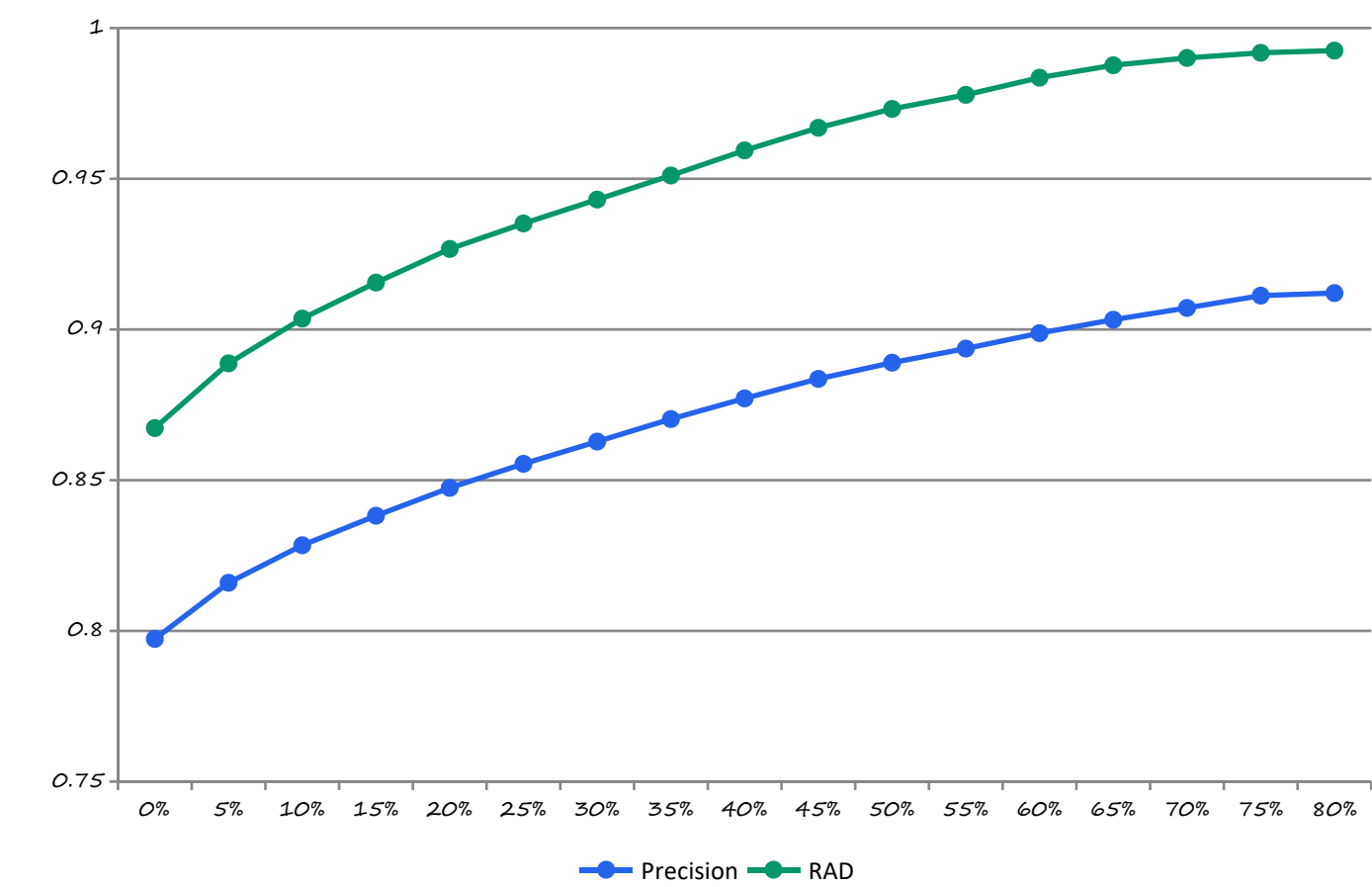


步骤 2: 机理 + *NN* + 分省份微调评估

最终指标来自保留测试子集上的分省份加权评估



结果：剔除比例越高，保留子集指标越高



“覆盖率—精度曲线”。

横轴为剔除测试集样本比例，保留的是纯饱和 *NN* 打分最高的测试样本，纵轴为精度。

故意伤害罪 结果表：
自由裁量指标达到 90% 目标需要剔除多少样本？

剔除比例	剔除样本	保留样本	Precision	RAD	结论
0%	0	17,518	79.7334%	86.7297%	RAD≥80 已达标
5%	876	16,642	81.5969%	88.8762%	继续上升
10%	1,752	15,766	82.8408%	90.3620%	RAD ≥ 90 首次达标
50%	8,759	8,759	88.8987%	97.3175%	保留一半样本
80%	14,014	3,504	91.2079%	99.2526%	仅保留 20%

注：这个split-mode为 global

交通肇事罪 结果表：
自由裁量指标达到 90% 目标需要剔除多少样本？

剔除比例	剔除样本	保留样本	Precision	RAD	结论
0%	0	6,560	78.4446%	84.3687%	RAD≥80 已达标
5%	328	6,232	82.6247%	88.8786%	继续上升
10%	656	5,904	84.0810%	90.5551%	RAD ≥ 90 首次达标
15%	984	5,576	84.9977%	91.6819%	保留85%
20%	1,312	5,248	85.9432%	92.8139%	保留 80%

注： 这个split-mode为 province

```

448
449
450 def main():  🐍 Ruby1R1ng *
451     parser = argparse.ArgumentParser(description="纯饱和 NN 筛选测试集后，再做机理+NN+分省份微调评估")
452     parser.add_argument("name_or_flags": "--data", default="merged_data_filtered_minor_0508_3.xlsx")
453     parser.add_argument("name_or_flags": "--province-col", default="省份/直辖市")
454     parser.add_argument("name_or_flags": "--time-col", default="判决时间")
455     parser.add_argument("name_or_flags": "--start-date", default="2021-01-01")
456     parser.add_argument("name_or_flags": "--split-mode", choices=["global", "province"], default="global")
457     parser.add_argument("name_or_flags": "--screen-by", choices=["oracle_precision", "confidence"], default="oracle_precision")
458     parser.add_argument("name_or_flags": "--target-metric", choices=["precision", "rad"], default="rad")
459     parser.add_argument("name_or_flags": "--train-ratio", type=float, default=0.6)
460     parser.add_argument("name_or_flags": "--val-ratio", type=float, default=0.2)
461     parser.add_argument("name_or_flags": "--min-samples", type=int, default=1)
462     parser.add_argument("name_or_flags": "--screen-epochs", type=int, default=30)
463     parser.add_argument("name_or_flags": "--base-epochs", type=int, default=30)
464     parser.add_argument("name_or_flags": "--finetune-epochs", type=int, default=30)
465     parser.add_argument("name_or_flags": "--train-batch-size", type=int, default=245)
466     parser.add_argument("name_or_flags": "--finetune-batch-size", type=int, default=245)
467     parser.add_argument("name_or_flags": "--lr", type=float, default=0.001)
468     parser.add_argument("name_or_flags": "--alpha-penalty", type=float, default=1.4)
469     parser.add_argument("name_or_flags": "--target-precision", type=float, default=0.80)
470     parser.add_argument("name_or_flags": "--max-remove-ratio", type=float, default=0.80)
471     parser.add_argument("name_or_flags": "--step", type=float, default=0.05)
472     parser.add_argument("name_or_flags": "--seed", type=int, default=42)
473     parser.add_argument("name_or_flags": "--output-dir", default=os.path.join("outputs", "screened_testset_experiment"))
474     args = parser.parse_args()
475     run(args)

```

screened_testset_experiment.py 里有个 split-mode，是数据集的划分方式

如果是global 就是全部打乱 随机6：2：2划分训练验证测试

如果默认换成province 就是按照每个省份的判决时间顺序 6：2：2划分 训练 验证 测试

在交通事故上测试了这两种方式 结果差不多 基本就是筛10%后，rad能到90%

但是如果是故意伤害，这两个mode一个是筛10% 一个需要筛15%， rad能到90%

SNN+fixed 自适应方法 vs 分省份微调

公平比较实验结果汇报 ·

31 个省份

同一测试集

先预测后更新

主对比方法

SNN+fixed 饱和在线

自适应流程：测试时先预测，再用当前标签更新状态

对照方法

分省份微调

按验证集选择策略后，在同一省份测试集上评价

评价指标

RA/ RAD

RA 为相对误差均值；RAD 为在自由裁量指标

全国测试集加权精度：分省份微调调整体更高

按各省测试样本数加权；主对比为 SNN+fixed linear 在线更新 vs 分省份微调。

方法	RA	RAD
SNN_fixed_linear	0.786972	0.863677
SNN_fixed_饱和	0.783575	0.861175
SNN_fixed_linear_在线更新（主对比）	0.788977	0.866165
SNN_fixed_饱和_在线更新	0.785061	0.863102
分省份微调（验证选择策略）	0.802465	0.869976

RA提升

+0.013488

分省份微调 - SNN+fixed linear 在线

RAD 提升

+0.003811

分省份微调 - SNN+fixed linear 在线

总测试样本

17531

31 个省份参与加权统计

省份胜出数

26 / 31

RA 下分省份微调高于主对比 SNN

结论

主对比下，分省份微调在全国加权 RA 与 RAD 上均高于 SNN+fixed linear 在线更新；但 SNN+fixed linear 在线更新在福建、山东、北京、海南、陕西等少数省份具有优势。

各省份主对比精度表（1-8 / 31）

SNN 列为 SNN+fixed linear 在线更新；微调列为分省份微调验证选择策略。Δ=微调-SNN。

省份	n	SNN RA	SNN RAD	微调 RA	微调 RAD	Δ RA	Δ RAD
四川省	455	0.778558	0.867685	0.814072	0.879976	+0.035514	+0.012291
广西壮族自治区	358	0.792709	0.867730	0.814490	0.879828	+0.021781	+0.012098
福建省	618	0.817245	0.902157	0.811968	0.886479	-0.005277	-0.015678
广东省	1206	0.791404	0.869712	0.799496	0.867031	+0.008092	-0.002681
山东省	1674	0.830228	0.899916	0.804818	0.870223	-0.025410	-0.029693
贵州省	378	0.735681	0.799321	0.770943	0.832066	+0.035262	+0.032745
安徽省	794	0.825413	0.910038	0.839620	0.908490	+0.014207	-0.001548
江西省	425	0.773325	0.860862	0.812331	0.873519	+0.039006	+0.012657

绿色 Δ 表示分省份微调高于 SNN+fixed linear 在线；红色表示 SNN 更高。

各省份主对比精度表（9–16 / 31）

SNN 列为 SNN+fixed linear 在线更新；微调列为分省份微调验证选择策略。 Δ =微调–SNN。

省份	n	SNN RA	SNN RAD	微调 RA	微调 RAD	Δ RA	Δ RAD
湖北省	769	0.768243	0.848911	0.795481	0.859389	+0.027238	+0.010478
辽宁省	926	0.774982	0.856052	0.795698	0.870845	+0.020716	+0.014793
吉林省	409	0.770614	0.836500	0.778236	0.836650	+0.007622	+0.000150
浙江省	726	0.803348	0.878982	0.815656	0.887597	+0.012308	+0.008615
湖南省	706	0.748517	0.827428	0.779513	0.833781	+0.030996	+0.006353
河北省	1618	0.808059	0.887585	0.820972	0.894384	+0.012913	+0.006799
山西省	390	0.785955	0.862217	0.798463	0.863786	+0.012508	+0.001569
北京市	484	0.808875	0.884560	0.800834	0.866536	-0.008041	-0.018024

绿色 Δ 表示分省份微调高于 SNN+fixed linear 在线；红色表示 SNN 更高。

各省份主对比精度表（17-24 / 31）

SNN 列为 SNN+fixed linear 在线更新；微调列为分省份微调验证选择策略。 Δ =微调-SNN。

省份	n	SNN RA	SNN RAD	微调 RA	微调 RAD	Δ RA	Δ RAD
江苏省	681	0.802606	0.885961	0.830184	0.904976	+0.027578	+0.019015
云南省	973	0.792669	0.881332	0.816988	0.886516	+0.024319	+0.005184
河南省	1408	0.759292	0.823943	0.779789	0.842004	+0.020497	+0.018061
甘肃省	469	0.792473	0.870248	0.802978	0.883094	+0.010505	+0.012846
黑龙江省	398	0.721794	0.778813	0.763926	0.821506	+0.042132	+0.042693
上海市	267	0.741876	0.818362	0.750167	0.814570	+0.008291	-0.003792
天津市	293	0.783518	0.872757	0.806448	0.869967	+0.022930	-0.002790
内蒙古自治区	298	0.773726	0.846115	0.775705	0.851592	+0.001979	+0.005477

绿色 Δ 表示分省份微调高于 SNN+fixed linear 在线；红色表示 SNN 更高。

各省份主对比精度表（25–31 / 31）

SNN 列为 SNN+fixed linear 在线更新；微调列为分省份微调验证选择策略。 Δ =微调–SNN。

省份	n	SNN RA	SNN RAD	微调 RA	微调 RAD	Δ RA	Δ RAD
青海省	109	0.782244	0.865515	0.802281	0.880205	+0.020037	+0.014690
重庆市	150	0.769758	0.858174	0.820726	0.875117	+0.050968	+0.016943
海南省	50	0.817623	0.881848	0.812268	0.860229	-0.005355	-0.021619
陕西省	309	0.799410	0.870999	0.779431	0.852283	-0.019979	-0.018716
宁夏回族自治区	108	0.817973	0.909346	0.832757	0.902463	+0.014784	-0.006883
新疆维吾尔自治区	58	0.724487	0.785971	0.764157	0.837727	+0.039670	+0.051756
西藏自治区	24	0.761696	0.826700	0.828439	0.900292	+0.066743	+0.073592

绿色 Δ 表示分省份微调高于 SNN+fixed linear 在线；红色表示 SNN 更高。

各方法全国加权结果汇总

保留 SNN+fixed 的 linear / 饱和 / 在线更新版本，便于复现实验输出。

方法	RA	RAD	相对主对比 Δ RA	相对主对比 Δ RAD
SNN_fixed_linear	0.786972	0.863677	-0.002005	-0.002488
SNN_fixed_饱和	0.783575	0.861175	-0.005402	-0.004990
SNN_fixed_linear_在线更新	0.788977	0.866165	+0.000000	+0.000000
SNN_fixed_饱和_在线更新	0.785061	0.863102	-0.003916	-0.003063
分省份微调_验证选择策略	0.802465	0.869976	+0.013488	+0.003811